

Block Assembler Robot

Firdavs Nasriddinov & Zachary Pestrikov & Sebastian Banuelos
ME/CS/EE 134: Robotic Systems
Professor Günter Niemeyer
California Institute of Technology

March 19, 2026

Video: [Drive](#)
Presentation: [Slides](#)

1 Task and Basic Concept

This project presents an autonomous "Block Assembler Robot" with the task of building a structure of magnetic blocks on top of a 6x6 grid of "base blocks". The structure is not hard coded, but rather prespecified by a JSON file (an example is given in Appendix A – Section 9), which contains the grid coordinates and colors of every block in the structure. The robot is able to go from almost any existing structure state to almost any JSON-specified structure state. This invariance of initial state means that the system is robust to most user actions including users adding and removing blocks from the structure or connecting and moving blocks around in the pile because the system is memory-less and therefore determines the next action to get the structure state closer to the desired final state every iteration of adding/removing a block.

Due to the open-ended nature of the structure the robot will be asked to build, there are a number of challenges risen by the task. The primary difficulties include accurately determining the state of the structure at any state despite the fact that some blocks may be obscured by others, as well as robustly recovering from missteps (finding misplaced blocks and placing them where they are intended to go) without further damaging the structure.

2 Mechanism and Hardware

The block assembly task requires a manipulator capable of reaching arbitrary positions on a planar workspace with sufficient dexterity to grasp, tilt, and rotate blocks during pick-and-place and separation maneuvers. We designed a 5-DOF arm driven by 6 HEBI motors with skeletonized aluminum links (14" shoulder-to-elbow, 12" elbow-to-wrist), a bicycle-cable-actuated two-finger gripper, and a wrist-mounted Intel RealSense depth camera.

2.1 Main Robot Hardware

The design consist of a 5DoF manipulator arm with 6 motors and a versatile two-finger gripper as an end-effector. The main structural frame utilizes two aluminum brackets (14" for shoulder-elbow link and 12" for elbow-wrist link) that have been skeletonized to minimize mass and inertia. The gripper is actuated via a motor near the shoulder motor that pulls on the gripper through a bicycle cable. The wrist-tilt motor is attached to a wrist bracket which supports the wrist-roll motor and the end-effector camera mount. We mount a RealSense depth camera to the camera mount at a vertical orientation with an angle that allows for viewing the current gripped block and scanning the workspace when the gripper is open and not obstructing the view.

Each motor was selected according to its expected maximum torque requirements when the arm is fully extended (Table 1). The five degrees of freedom—three Cartesian axes plus tilt and roll—were chosen as the minimum set needed for the task: tilt enables the separation maneuver that slides magnetically-attached neighbors off, and roll aligns the fingers with a block's edges at any orientation. Yaw is unnecessary because all blocks are approached from above.

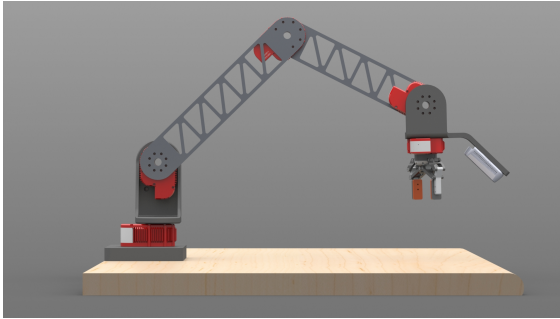


Figure 1: Side View

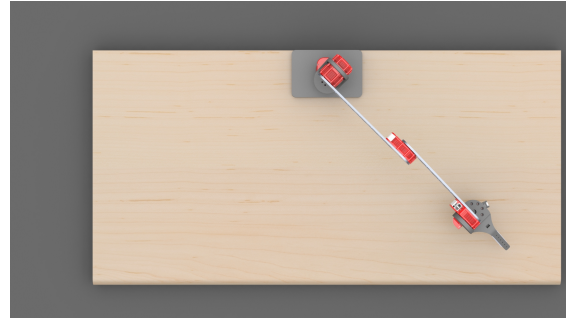


Figure 2: Top View

Table 1: Robot Actuator Torque Specifications

Joint	Motor	Peak (N·m)	Cont. (N·m)	Max Load (N·m)
Base	X8-9	20.0	8.0	Not Under Load
Shoulder	X8-9	20.0	8.0	7.36
Elbow	X5-4	7.0	4.0	2.38
Wrist Tilt	X5-1	2.5	1.3	0.1
Wrist Roll	X5-1	2.5	1.3	Not Under Load
Gripper	X5-1	2.5	1.3	0.1

2.2 Gripping

The design incorporates a two-finger parallel gripper used to pick up and place blocks. The gripper is actuated by a HEBI X5-1 motor mounted near the shoulder, which pulls on the gripper fingers through a bicycle cable routed along the arm. Placing the motor near the base rather than at the wrist significantly reduces distal mass and inertia, improving the arm's dynamic performance and reducing the torque requirements on the wrist and elbow joints. The motor's rotary position is converted to a linear finger opening via a simple radians-to-millimeters mapping, with the fingers moving approximately 15 mm forward when fully closed. The inner faces of the fingers are lined with a foam material to increase friction during grasping, preventing blocks from slipping during transport and separation maneuvers.

2.3 Cameras

The system uses two cameras. A Logitech webcam is mounted in a fixed elevated position above the workspace, providing a top-down view. An Intel RealSense D435i depth camera is mounted to the wrist bracket above the gripper, oriented vertically to view the grip area and the workspace below. Further details on the detection pipelines and calibration procedures for each camera are described in Section 4.

3 Task Space, Kinematics, Trajectories, Precision, Continuity (Basic Robotics)

3.1 Task Coordinates

Our task coordinates are the x, y, z coordinates of the tip of the gripper when it is fully closed and the gripper’s tilt angle, θ_{tilt} , relative to the table and the gripper’s roll angle when facing the table θ_{roll} . The tilt angle is measured relative to when the gripper is facing away from the table, which means that when the gripper is parallel with the table it is at 0 radians when facing along the table and $-\pi$ radians when facing towards it. The roll angle is relevant only when the gripper is facing down towards blocks on the table in preparation to pick or place them. The zero position of the gripper motor is when the line between its fingers is parallel with the x axis and the axis that the gripper rotates about being the negative world z axis in this position.

3.2 Kinematics

The tilt angle is simply computed in forward kinematics as the sum of the shoulder, elbow, and wrist tilt motors each multiplied by the sign that they move in (with the elbow being negative and the others being positive). Similarly, the roll angle is computed as the sum of the negative base angle and positive wrist roll motor angle. Therefore, the Jacobian for the last two task coordinates (θ_{tilt} and θ_{roll}) are simply rows with ones or negative ones depending on direction on the joints that contribute to the angle. This results in the following matrix:

$$\begin{bmatrix} 0 & 1 & -1 & 1 & 0 \\ -1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

We simply add these two rows to the bottom of the cartesian (x, y, z) Jacobian to compute those last two task entries.

The inverse kinematics is computed via the Newton-Raphson algorithm. For the Jacobian, we only use the first four task coordinates – the cartesian coordinates and gripper tilt – as the task vector and ignore the Jacobian’s last row. This is because the gripper roll task is only useful when the gripper is pointing straight down towards the table and its degree of freedom corresponds directly to the gripper roll motor, so we can simply solve for it after the inverse kinematics as $q_{gripper_roll} = \theta_{roll} + q_{base}$ based on the desired roll angle and the base motor’s state outputted by the Newton-Raphson.

3.3 Trajectories

The arm’s motion is governed by a trajectory queue—an ordered sequence of waypoints, each specifying a target joint configuration with velocity, a movement duration, and optional hold delays before and after the motion. The Manipulator node interpolates between consecutive waypoints using fifth-order polynomial splines, which guarantee continuous position and velocity with zero boundary accelerations for smooth motion. The Brain sends trajectory commands with a type field—replace, append-front, or append-back—that controls how new

waypoints merge with any in-progress motion. This allows the system to chain complex multi-step maneuvers (e.g., grasp, lift, separate, grip-check, approach, place) as a continuous trajectory without stopping between phases.

3.4 Regularization

To ensure our robot was stable in the real-workspace, we imposed joint limits, used a damped pseudoinverse, and "restricted" to ensure elbow-up solutions. We limited the wrist-tilt motor to be $\in [-\frac{5\pi}{4}, \frac{\pi}{4}]$ to ensure we don't impose a position on the motor such that neither the wrist-bracket collides ($-\frac{5\pi}{4}$) nor the camera mount collide with the upperarm link. We used a damped pseudo-inverse calculation to compute the inverse of the Jacobian via SVD, which ensures singular values below parameter gamma are regularised to avoid numerical instability near kinematic singularities. During early testing we noticed that sometimes the inverse kinematics solution would have an elbow-down configuration, which adds the risk of having the elbow motor colliding with the table. To account for this, we made our robot workspace slightly more conservative and made each joint guess to Newton-Raphson start from an elbow-up solution. From this point on, we didn't observe any elbow down ikin solutions.

3.5 Gravity Compensation

The gravity compensation model for the robot arm is formulated as a multi-body static-torque system. The model calculates the torques required at the shoulder, elbow, and wrist-tilt joints to counteract gravitational loads, assuming a vertical zero-configuration. The remaining joints (base, wrist-roll, and gripper) are assumed to be invariant to gravitational loading in the arm because they do not affect the extension of the arm and can not oppose the torque caused by gravity.

The model utilizes precomputed moment coefficients, C_i , for each joint. These coefficients aggregate the moment generated by a link's own mass (m_i) acting at its center of mass (d_i) and the moment generated by the total distal mass (W_{distal}) acting at the link's full length (L_i). This relationship is defined as:

$$C_i = g(m_i d_i + W_{distal} L_i) \quad (1)$$

where g represents the acceleration due to gravity. An empirical tuning factor is included in the distal mass of the elbow to account for unmodeled hardware and mounting components, ensuring the system does not undershoot during compensation.

The required torques are calculated recursively from the end-effector back to the base. Let θ_1 , θ_{12} , and θ_{123} represent the absolute angles of the shoulder, elbow, and wrist relative to the vertical axis. The gravitational torques (τ_g) are determined by:

$$\tau_{g,w} = C_3 \sin(\theta_{123}) \quad (2)$$

$$\tau_{g,e} = C_2 \sin(\theta_{12}) + \tau_{g,w} \quad (3)$$

$$\tau_{g,s} = C_1 \sin(\theta_1) + \tau_{g,e} \quad (4)$$

The final commanded motor torques are derived by negating the calculated gravitational torques and adjusting for the specific rotational direction (d_i) of each actuator:

$$\tau_{motor,i} = -d_i \cdot \tau_{g,i} \quad (5)$$

3.6 Error Map

We produced two calibration error maps to compensate for systematic forward-kinematics inaccuracies. The XY error map (Figure 3) is a vector field showing the offset between commanded and actual tip positions across the workspace, measured during the XY calibration procedure described in Section 4.2.2. The Z error map (Figure 4) is a heatmap of measured table height versus radial distance from the base, captured during the Z calibration described in Section 4.2.3. Both maps are stored as CSV files and loaded by the IK Solver at startup. During each IK solve, the solver interpolates from these maps to apply position-dependent corrections to the task coordinates before computing joint angles, compensating for systematic FK errors that vary with arm configuration.

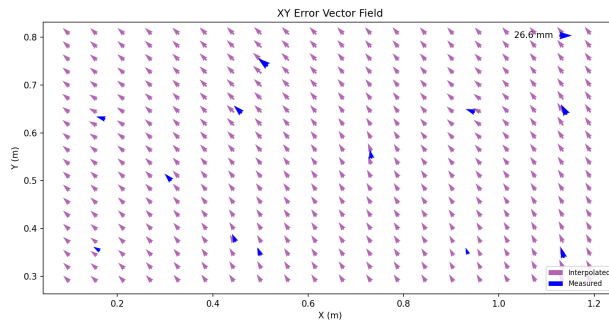


Figure 3: XY Error Map

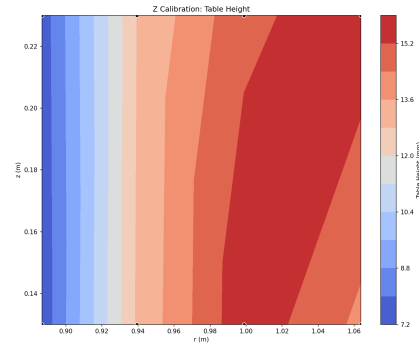


Figure 4: Z Error Map

3.7 Motion Detectors

We implemented two simple motion detectors to determine contact collisions, gripper failures, and jerk movements.

To detect collisions, we check for the difference between the commanded and actual joint positions, velocities, and effort and determine a collision occurred if any is above a threshold. We made the threshold be proportional to the speed of the robot which serves as a heuristic for incorporating inertia to our collision check.

To determine a gripper failure, we check for the magnitude of the effort exerted by the gripper motor to achieve the commanded gripper position (fully closed) and signal a failure if the value is lower than a threshold. This is because if we are expecting a block to be gripped but

it actually isn't then the gripper will be fully closed and the motor will not need to exert as much effort to try to fully get to the closed position.

As we continued developing our robot and the software for it, our required computations grew. We started to notice that as more compute was being spent, the higher the chance or the HEBI motors to lose connection and thus "drop" to a lower position. After connection would be gained, the motors would realize the commanded joint positions and move back quickly with a noticeable jerky movement. To determine jerk movements, we do the same logic as contact collision check but only on the difference between commanded and actual joint positions. Note that after we determine a jerk movement, we don't send the joint command and simply store to history. After a certain amount of these joint position discrepancies occur, we add a trajectory waypoint to have the arm move to the correct joint position smoothly.

4 Cameras and Visual Detectors

4.1 Cameras

The system takes input from two cameras – one Logitech camera (the top camera) elevated in a fixed position above the robot and one RealSense depth camera attached to the robot right above the gripper. The top camera is primarily used for identifying blocks in the "pile" – a section of randomly assorted blocks to the right of the robot that it can use to build the structure – to be used to build the structure as well as identifying whether the robot accidentally picked up multiple blocks at once. The RealSense camera is used primarily when building the structure to determine the structure state as it can determine the z coordinate of the blocks in the structure, and can therefore tell us which layer the blocks in the structure are in.

4.2 Calibration

4.2.1 Aruco Markers

We produce a mapping of pixel-to-world from the top camera frames with the positioning of Aruco markers at the corners of the workspace both at the start of the program and periodically (typically every 100 seconds) as the robot runs in case the table shifts. The detector knows the world coordinates of the Aruco markers, so it uses the detected pixel coordinates of those markers to create an affine transform matrix to convert between pixel and world coordinates.

4.2.2 XY Calibration

We calibrate the robot's kinematics errors in the xy plane before the start of construction by having the robot move to various positions in the workspace, detecting the position of a blue marker on the robot's camera mount at those points with the top camera, and storing the offset between the expected position (i.e. the task command sent to the robot) and the actual position of the tip. These offsets are then used by the inverse kinematics solver to

offset the task coordinates by an amount computed by interpolating the offsets (Figure 3) between the different points that the calibration was done at.

It's important to note that since the blue mark on the camera mount is not flat with the table, we implemented a ray-traced-based way to convert pixel coordinates to world coordinates with accurate z mapping.

4.2.3 Z Calibration

We calibrate the robot's kinematics errors on the z -axis (caused primarily by imperfect gravity compensation) before the start of construction by having the robot move along the radial direction at different z positions and detecting the height of the table at those positions with the end effector camera. As the tip moves farther out in the radial direction, the table appears higher in world coordinates even though it is meant to be at a constant height of 0. This is because the imperfect gravity compensation has a greater effect as the robot extends out further (Figure 4). We store these table heights at different radii and apply an offset to task coordinates before initiating the inverse kinematics solve, similar to the xy calibration.

4.2.4 Placement Grid Calibration

The placement grid calibration determines the world-frame positions of the structure's base grid corners using the overhead camera. While the grid corner coordinates are initially measured by hand, this step re-detects them through the same camera and calibration pipeline used for the XY kinematics calibration. This ensures that the camera's understanding of where the grid is uses the same pixel-to-world transform as the kinematics error correction, preventing errors from different subsystems (camera calibration vs. kinematics) from compounding.

4.3 Block Detection: Pile

To detect potential blocks from the pile we use a simple contour detection algorithm for the top camera as the blocks we are detecting are always squares. We first filter by color, using HSV limits to define the colors corresponding to the blocks that we use. We then use the OpenCV library to isolate the contours of objects within those HSV limits. We then use a custom edge-aligned bounding box method, which identifies the edges of the contour and the angle of the longest edge, rotates the contour by this angle, computes the axis aligned bounding box of this rotated contour, rotates this axis aligned bounding box back to the countour's angle, and returns it. This method finds a bounding box for the contour in a manner that aligns with the actual blocks within it, and therefore allows us to identify where the blocks are within the contour. Here we have three cases: either the bounding box is smaller than the size of a typical block, in which case we discard it; the bounding box is approximately the size of a block, in which case we just detect it as a single block, or it is the size of at least two or more typical blocks, in which case we split it up into its internal blocks, which we mark as connected.

To split it up we use a custom "rectangle splitting algorithm", which when given one of these large bounding rectangles (detected by either the top or end effector camera), will split this

large bounding rectangle into individual detected blocks and determined which blocks within that rectangle are connected. The algorithm works by taking the bounding rectangle and splitting it into a grid based on the expected dimensions of a block in pixel space at the height of the camera. It then checks whether each square in that grid is occupied by a block and designates the corresponding block as connected to the other detected blocks adjacent to it in the grid if it is occupied. This allows us to detect which faces of any detected block are connected.

4.4 Block Detection: Structure

To determine the current completeness state of the structure, we use the point cloud (points specified in xyz with RGB color) generated by end-effector depth camera to accumulate an occupancy grid (which is explained later) which we can query to detect blocks in 3D. For each layer (z modulo block size), we query the occupancy map and filter by color using HSV limits. This gives us a layer-wise + color-wise binary map which we can use the exact algorithm as for the block detection from pile (equivalent to layer=0) to detect blocks from. Importantly, the structure detector operates in grid-relative coordinates: detected block positions are mapped relative to the base grid corners (via an affine transform matrix) rather than in absolute world coordinates. This decouples kinematics errors from the world-to-end-effector camera transform errors, as both the grid registration and the block detections pass through the same camera pipeline.

4.5 Grip & Connection Detection

In addition to object detection, we are able to validate whether we grabbed blocks properly with the two cameras. Using the end effector camera, we detect the depth of the grasped block between the gripper. This allows us to tell whether the block is angled or straight – if it is straight, then the depth is consistent across the block, but if it is angled then there are points that are higher or lower, which allows us to tell that it is angled. Importantly, we cannot simply use the depth sensor on the end-effector camera as the camera is too close to where the block would be gripped and thus out of the supported range for the camera. Instead, we use a pre-trained RGB-to-depth neural network model (Depth Anything V2) to produce a depth map which uses the RGB image from the same camera. See Appendix 11 for more details.

Additionally, after we pick up a block we bring it up to the top camera to determine whether there are any blocks that are attached to it (i.e. the block was improperly detected as not connected, our separation maneuver failed to separate blocks, or the block attached to another block as it was getting picked up). We determine this by simply bringing the block directly under the top camera of the robot and filtering for blocks that appear very large (take up 150 px) that are close to the center of the frame. This allows us to determine how many blocks are close to the top camera, which correspond to the number of blocks the robot has picked up. Note that we decided to use the top-camera for this detection as 1) it allowed us to use our existing, thoroughly-tested block detection algorithm and 2) the end effector camera has a limited and unpredictable view of the gripping area (especially for certain edge cases).

If the robot detects any mistakes in the grip or connections of the block it has picked up, it is able to recover from it (which will be explained later in this paper).

5 ROS Software Architecture

The legobuilder software is structured as four cooperating ROS 2 nodes that communicate exclusively through a publish-subscribe topic graph. The four nodes are the Brain, the Manipulator, the Detector, and the IK Solver. Every node additionally participates in a mutual heartbeat protocol, publishing a boolean on its own `<node>/heartbeat` topic at a fixed rate and monitoring peer heartbeats to detect node failures.

Figure 5 summarizes the primary inter-node message flows.

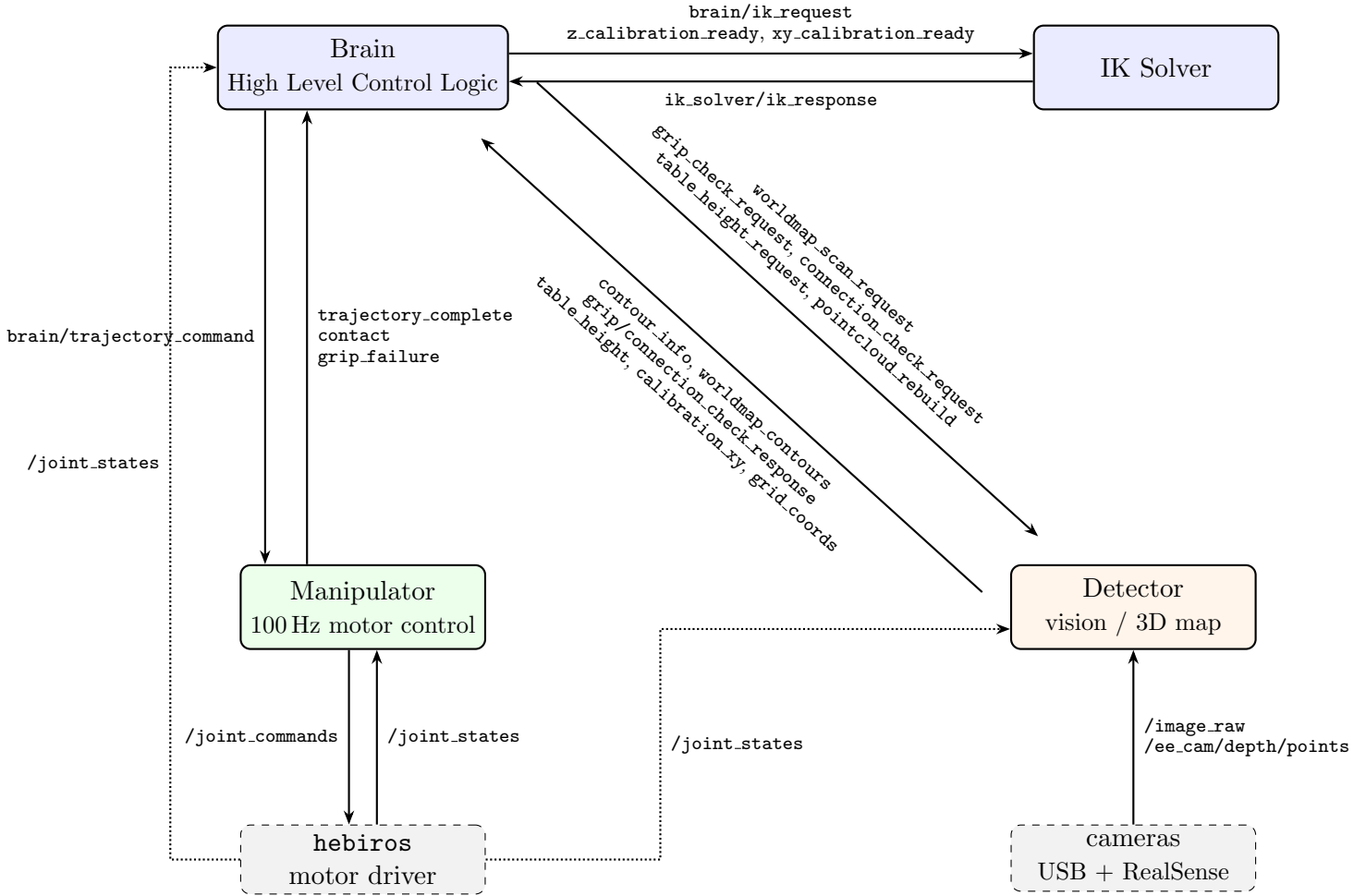


Figure 5: ROS 2 topic flows between the four legobuilder nodes and hardware interfaces. All topics are shown verbatim except heartbeats, which are omitted for clarity. Dotted lines indicate that `/joint_states` is also consumed by Brain and Detector for forward-kinematics lookups. Brain → IK Solver calibration signals (`z_calibration_ready`, `xy_calibration_ready`) prompt the solver to reload its offset tables from a stored csv file. The five Brain → Detector request topics cover world-map scanning, visual grip quality, block connection detection, table-height measurement (Z calibration), and point-cloud cache rebuilding. The Detector responds with contour detections (`contour_info`, `worldmap_contours`), check responses (`grip/connection_check_response`), and calibration results (`table_height`, `calibration_xy`, `grid_coords`).

5.1 Brain Node

The Brain is the central orchestrator of the system. It hosts the **build pipeline**—a three-phase state machine (grasp, approach, place) that sequences each block placement. The pipeline is supported by several dedicated subsystems: a **structure scanner** that manages overhead scans to reconstruct the current build state; a **block structure** model that maintains a 3D grid representation of the target design and tracks which cells have been filled; a **calibrator**

that runs the XY and Z calibration sequences at startup; and a **precompute manager** that can speculatively solve IK for the expected next trajectory while the arm is in motion.

The Brain operates primarily through callbacks rather than a periodic loop. Incoming block detections on `detector/contour_info` feed an **object processor** that temporally buffers and clusters detections using DBSCAN—chosen because it does not require specifying the number of clusters and naturally designates sparse detections as outliers—before publishing stable block positions. When the pipeline is ready to move, it publishes an IK request on `brain/ik_request` containing the full set of Cartesian waypoints, tilt and roll angles, and gripper states. On receiving the solved joint angles on `ik_solver/ik_response`, a callback assembles a trajectory command and publishes it to `brain/trajectory_command`. The pipeline then waits for a trajectory-complete, contact, or grip-failure message from the Manipulator. A grip failure—the Manipulator detecting via effort that no block was actually gripped—causes the Brain to discard the attempt and re-detect pile blocks. A **separator** plans trajectories to detach magnetically-connected blocks before grasping. After a successful grasp of a single block, a visual **grip check** via the Detector classifies grip quality; non-good results trigger recovery strategies (nudging for low/high grips, rotating for diagonal grips).

Additional subsystems include a **gesture manager** for idle behaviors (a "chomp" gesture to signal to the user that no blocks are available in the pile, and a "nod" gesture to request human help with unreachable blocks), a **dashboard client** that posts build-state events to an external web dashboard via fire-and-forget HTTP, and **reachability analysis** that determines which blocks near the structure can be reached by the gripper without colliding with the existing structure.

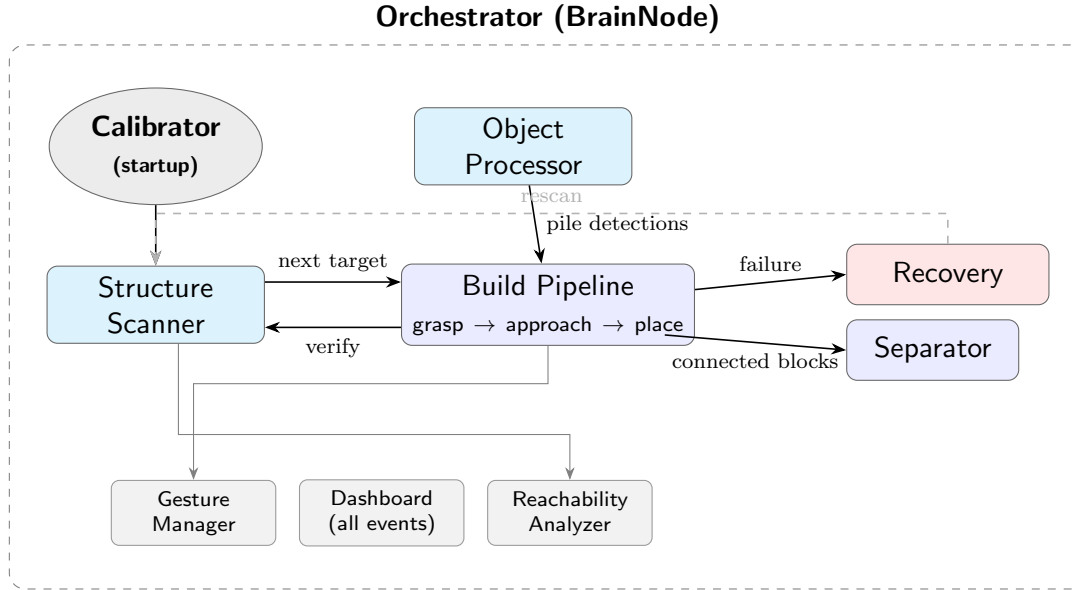


Figure 6: Brain node internal architecture. The Orchestrator (the **BrainNode** class) manages all subsystems. The calibrator runs at startup before the main loop begins. The build pipeline orchestrates each placement cycle, receiving pile detections from the object processor and the next target from the structure scanner. The separator detaches magnetically-connected blocks before grasping; on failure, recovery is invoked before rescanning. The gesture manager signals idle states to the user, the dashboard receives events from all subsystems, and the reachability analyzer informs the structure scanner which blocks near the structure are accessible.

5.2 Manipulator Node

The Manipulator is the only node with a hard real-time periodic loop. A 100 Hz timer interpolates the current **trajectory queue**—an ordered sequence of waypoints, each specifying target joint positions, a movement duration, and optional hold delays. Interpolation uses quintic polynomial splines for smooth position and velocity profiles with zero boundary accelerations. The node applies gravity compensation torques and publishes the result to `/joint_commands` for the motor driver. A second publisher sends the same state to `/joint_states_viz` with the gripper position converted to meters for RViz display. Trajectory input arrives asynchronously through a callback on `brain/trajectory_command`, which carries an array of waypoints along with a command type (replace, append-front, or append-back) that controls how new waypoints merge with any in-progress motion.

At each 100 Hz tick, the Manipulator also compares actual joint states (received on `/joint_states`) against the interpolated reference for three checks: **collision detection**—position, velocity, or effort errors exceeding thresholds proportional to robot speed trigger an immediate contact message; **grip failure detection**—when the gripper is commanded fully closed, the node checks whether the gripper motor’s effort falls below a threshold; if it does, the block was not actually gripped, because a block between the fingers would resist closure and require the motor to exert more effort, so a grip failure message is published; and **jerk prevention**—large sudden position discrepancies caused by brief HEBI motor connection drops are detected,

and the erroneous joint command is suppressed rather than forwarded. After repeated jerk occurrences, the node inserts a smooth recovery waypoint to return the arm to the correct position without abrupt motion.

5.3 Detector Node

The Detector manages two independent perception pipelines. The first is the overhead camera pipeline: whenever a new frame arrives on `/image_raw` (published by the camera at roughly 15 FPS with a queue size of one, so stale frames are dropped), a callback runs HSV segmentation, contour extraction, and shape classification to identify blocks. Detected contours are projected into world coordinates and published on `detector/contour_info`.

The second pipeline handles the wrist-mounted RealSense depth camera. Raw point clouds arrive continuously on `/ee_cam/depth/points` and are buffered internally. When the Brain issues a scan request on `brain/worldmap_scan_request`, a callback transforms the buffered point cloud into world coordinates using the current arm FK state (tracked via `/joint_states`) and integrates it into a probabilistic 3D voxel map. The integrated map is published as a point cloud on `detector/world_map`; if the scan type requests block detection, an array of detected contours is also published on `detector/worldmap_contours` for the Brain's structure verification logic. The detector node additionally handles request-response interactions for visual grip quality assessment and for connection detection after placement. We should note that the Depth Anything model is hosted on a separate process which is served to the detector via a FastAPI app. Calibration measurements—table height, XY position, and grid corner coordinates—are handled by the detector node through a similar set of request-triggered callbacks.

5.4 IK Solver Node

The IK Solver exists specifically to offload numerically intensive inverse kinematics computation to a separate OS process, preventing IK solve time from blocking the Brain's callback queue. The node has no periodic control loop; it is entirely event-driven. Each incoming solve request message on `brain/ik_request` spawns a background daemon thread that solves IK for all N waypoints in the request, applies pre-computed XY and Z calibration offsets via distance-weighted interpolation, and publishes a response message on `ik_solver/ik_response` carrying the flattened $N \times 6$ joint angle solutions and the original request ID.

Requests carry a type field distinguishing "primary" requests (needed immediately to execute a motion) from "precompute" requests, which are speculative requests of the expected next trajectory in the robot's nominal flow that will be executed if no recovery behaviours are needed. A primary request will preempt any in-progress precompute thread. The IK Solver also listens for `brain/z_calibration_ready` and `brain/xy_calibration_ready` signals, which prompt it to reload the corresponding offset tables from stored csv files so that subsequent solves immediately reflect the new calibration.

6 Sensing/Thinking: Modeling/Understanding the World

6.1 Buffering Top Camera Detections

In order to accurately detect objects from the pile, the brain keeps track of "pile detection buffer", which stores every block detection in the pile over the past t_{buffer} seconds (t_{buffer} is a constant in the code, which is typically set to approximately 2 seconds). We then perform a clustering algorithm (using scikit-learn implementation of DBSCAN, which groups dense data points and designates sparse points as outliers and doesn't require specifying the number of clusters) on the detections stored in the buffer to determine which detections are of the same object. If and only if the average distance between the detections of the block's position and the average position of the detections is within a certain tolerance (approximately 2 cm) and we have consistently detected the block for the duration of the buffer, then we consider those detections to correspond to an actual detected block that the robot can use, where the position and orientation of that block is the average of all the positions and orientations detected in the buffer.

This buffering algorithm prevents our system from prematurely publishing the position of an object if it is still moving or only in the frame for a brief moment, and requires that the object be stationary and in the frame for t_{buffer} seconds.

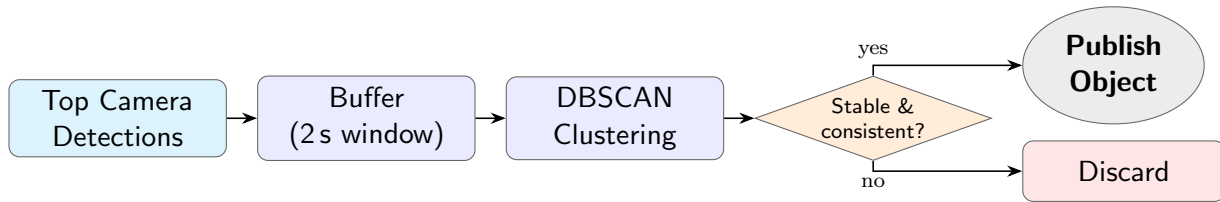


Figure 7: Top camera detection buffering. Raw detections are accumulated over a sliding window, clustered with DBSCAN, and published only if the cluster is spatially stable and consistently detected for the full buffer duration.

6.2 World Map Construction

In order to construct an accurate representation of the world the brain combines point clouds detected from the end effector camera into a robust world map. As we traverse the workspace gathering depth information we construct a 3D world reconstruction, which gives us the ability to actually determine the location of blocks within the structure, which is 3D.

It works via an implementation of an occupancy map that splits the workspace into voxels and computes the log odds ratio that each voxel is occupied by a color. Every time the detector receives a frame from the end effector camera, the state of a voxel is updated based on whether the camera measured a block in that voxel on that frame. This gives us a probability that the block is actually there rather than a binary state. In detail, we store the occupancy map in a sparse voxel grid with Bayesian occupancy, decay, and frustum miss. Storage is keyed by integer (ix, iy, iz) tuples derived from world-frame coordinates. Parallel arrays hold per-voxel state: log-odds occupancy, observation count, last-seen scan index, and RGB

color. Each time the brain requests a scan, the detector runs the full probabilistic pipeline to integrate all detections:

1. Decay – multiplicative confidence decay on stale positive log-odds
2. Hit update – log-odds addition with clamping to $[L_MIN, L_MAX]$ and observation count capped at `obs_count_max`
3. Frustum miss – penalty for confident voxels in the camera’s field of view but not observed this scan
4. Pruning – periodic garbage collection of dead voxels (near `L_MIN`)

The grid doubles its backing arrays on demand so that typical integrations involve only bulk numpy operations plus a dict-membership check over the (usually 1-10 K) unique voxels per frame to minimize compute needed.

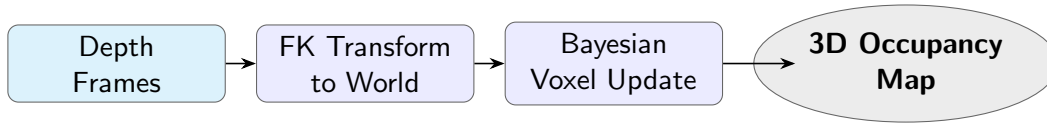


Figure 8: World map construction pipeline. Each depth frame is transformed into world coordinates via the current world-to-depth-camera FK state, then integrated into a sparse voxel grid through four steps: confidence decay on stale voxels, log-odds hit updates, frustum-miss penalties for unobserved voxels in the camera’s field of view, and periodic pruning of dead voxels.

6.3 Structure Reconstruction

At the start of construction and after every block is placed, the system reconstructs the state of the structure internally to determine what the next block to place should be. To do this, the arm moves the camera to a position directly above the structure and integrates several scans of the end effector camera to construct a world map, which contains positions of blocks in the structure. The system then matches the positions of the blocks in the structure to grid cells defined by the row, column, and layer relative to the base grid of blocks. It is then able to determine whether the detected blocks are in the correct positions. If the detected blocks are obscuring any other blocks (i.e. if there are several blocks stacked on top of each other, then the detector will only see the top block due to it only accumulating pointclouds from a top-to-bottom POV), then it assumes that the obscured blocks are in the correct place. This assignment of blocks to their cells allows the system to maintain an inner understanding of what the structure is, and therefore allows it to determine what the next block to place or remove should be.

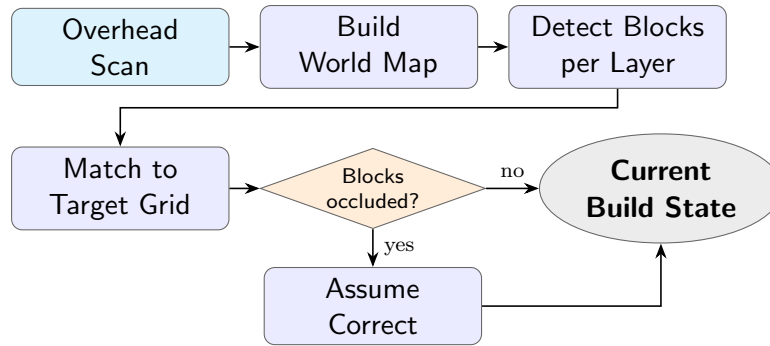


Figure 9: Structure reconstruction. The arm scans from overhead, integrates depth data into the world map, detects blocks layer by layer, and matches them to the target grid. Blocks hidden beneath visible layers are assumed to be correctly placed.

7 Action: Normal Behaviors, Failure Recoveries, and other Special Features

The system constructs a target block assembly by repeating a perception–plan–act cycle: it scans the structure to determine the current build state, selects the next block to place (either from the disorganized pile or a misplaced block near the structure), grasps it, verifies the grip, carries the block to the target location, and places it. Each iteration of this loop involves a fixed set of named behaviors, organized below by category. Figure 13 shows the flow between these behaviors.

7.1 Scan Behaviors

Structure observation. At the start of each placement cycle—and periodically when no work is available—the arm moves to a fixed overhead position above the build area and dwells there while the top camera accumulates depth and color data.



Figure 10: Scanning pose to observe the current state of the structure.

This data is used to reconstruct a three-dimensional occupancy model of the partially built structure, identifying which cells are already filled and with what color. If the overhead view recovers fewer than half the blocks expected to be visible (typically due to occlusion from tall or overhanging pieces), the arm automatically falls back to an *orbit scan*: it moves sequentially through a ring of side-angle viewpoints surrounding the structure, capturing depth images at each stop. In our final demo configuration, the orbit radius was set to zero, resulting in repeated scans from the same overhead viewpoint; the system is designed to support a nonzero orbit radius for multi-angle coverage. Detections from all viewpoints are pooled and de-duplicated to yield a more complete structural estimate. This scan result determines both the next target cell to fill and whether any placed block is the wrong color.

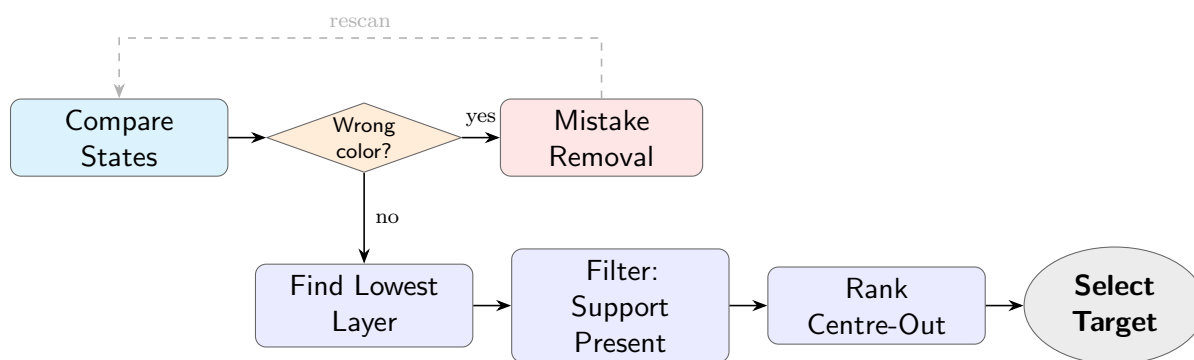


Figure 11: Next target selection. After a structure scan, the current build state is compared against the target design. If a wrong-color block occupies a target cell, the system performs a full mistake removal cycle—grasping the misplaced block from the structure, carrying it to the pile, and rescanning—before re-entering the selection logic. Otherwise the system finds the lowest incomplete layer, filters for cells with support, ranks candidates from the centre outward, and selects the next target.

Grip check. After every grasp, before the block is transported, the gripper aligns the block directly beneath the end effector camera. An RGB image of the grip area and held block is captured and sent to an external depth-estimation server. The resulting depth map is analyzed across three regions of interest spanning the two fingers and block. The system classifies the grip into one of five states: *good* (block centered and level between the fingers); *low* (block has slipped too deep, with insufficient protrusion); *high* (block protrudes excessively above the fingers); *diagonal* (block is twisted roughly 45° relative to the finger axis); or *partial* (block is gripped well but not perfectly). Any state other than good triggers a recovery action (the Grip Recovery behaviour described below) before the arm proceeds.

Connection check. Following a successful grip check, the arm moves to an inspection pose where the gripper is directly below the top camera, which counts the number of blocks visible in the immediate vicinity of the gripper. The connection check also verifies that the held block's color matches the target cell's expected color, rejecting color mismatches before placement. If only one block is detected, the grasp is clean and the arm proceeds to the target. If two or more blocks appear, one or more extra blocks have been inadvertently lifted by magnetic attraction on accident. In this case the arm discards all blocks into the pile and returns to idle so the blocks can be re-detected in their new positions for the arm to grasp later. Further, if no blocks appear (signaling a failed connection check), the arm discards the block(s) and restarts the grasp phase as it's not confident in the block(s) it may have grasped.

7.2 Grasping and Manipulation Behaviors

Block grasping. When a target pile block is selected, the arm approaches it from directly above along a vertical descent. The wrist is oriented along the edges of the block that minimize the chance of the gripper fingers bumping into any nearby blocks while descending. The arm descends, closes the gripper, and lifts the block clear before proceeding to the grip check.

Block separation. Because the blocks connect magnetically, a block in the pile may be attached to one or more neighbors. Simply lifting it would drag those neighbors up with the block. The system detects connected faces each block in the pile is connected to and separates all connected blocks before grabbing the target block.

For a block with a single connection or for a pile block, *in-place separation* is used: after lifting the block to a safe height, the wrist tilts by 45° relative to vertical and the arm lowers slowly, pressing the attached neighbor back down against the table. This causes the connected block to be slid off of the target block, yielding just the target block in the grippers fingers. The system will then proceed to a grip check. If a block is not immediately graspable (i.e. it has more than one attached face), then the robot will separate the extra connected blocks from the set one-by-one by grabbing and separating them until the target block has only one connection remaining, then it will grab the target block and separate it from its final

connected block and proceed to a grip check. The extra blocks are placed in the most wide open area of the pile – far from the target block to prevent them from reconnecting.

Mistake removal. When a structure scan reveals that a block of the wrong color occupies a target cell, the arm treats the misplaced block as a special pickup target: it grasps the block directly from its position on the structure, carries it to the most wide open area of the pile, and sets it down gently. The structure scan is then repeated to reassess the build state before normal placement resumes.

Placement. Once a block has been grasped, verified, and carried to the target cell, the arm descends to the placement position along a specific vector with the gripper closed, opens the gripper to release the block (pausing briefly to allow the magnetic connection to seat), and retracts upward. The vector of approach is determined as the average of the negative unit normal vectors of the faces the block will be attaching to and a unit vector pointing straight down. This allows the gripper to approach the placement position in a way that minimizes the chance that it interferes with surrounding blocks while also maximizing the chance the block actually connects to the faces that it is intended to connect to. If the block is connecting to more than one face, then the gripper moves to a slightly higher placement position and drops the block into place to prevent the gripper fingers interfering with its neighbor blocks rather than placing it into its spot, which would cause the gripper fingers to bump the block's neighbors.

The wrist roll during approach and descent is chosen according to which horizontal neighbors of the target cell are already in place, so that the fingers clear any surrounding structure. After each placement the arm performs a structure observation scan to confirm the block is seated correctly before advancing to the next cycle.

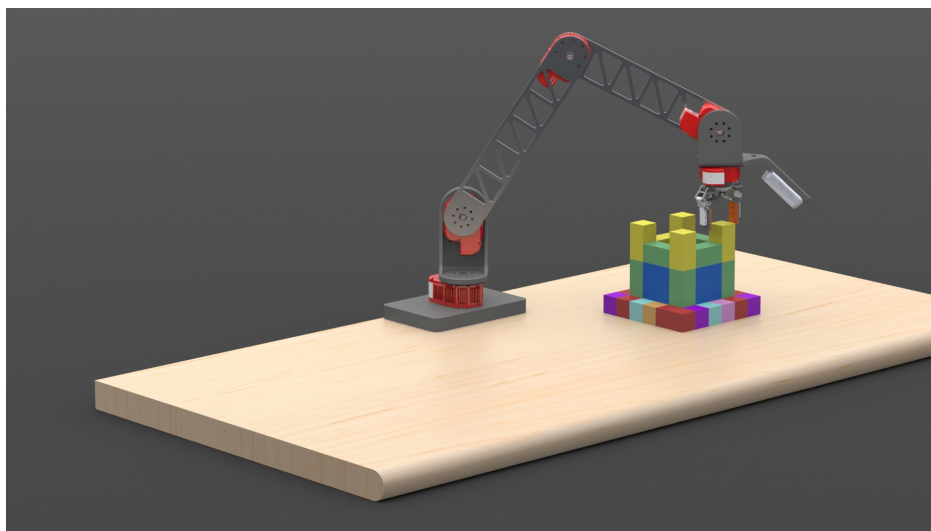


Figure 12: View of the robot placing a block on the structure.

Grip recovery. When the grip check returns a non-good classification, the system attempts recovery before proceeding. For a *low/high* grip—where the block has slipped too deep or protrudes excessively above the fingers—the arm re-grasps by nudging the pick position radially outward/inward by 10 mm per retry attempt, biasing the grip higher on the block face. For a *diagonal* or *partial* grip—where the block is misaligned in the fingers—the arm rotates the wrist by 45° and re-grasps at the same position, changing which face of the block contacts the fingers. Both strategies are retried up to two times; if the grip remains unacceptable after the maximum retries, the block is released and the arm returns to idle to select a different block.

7.3 Main Loop Flow

Figure 13 illustrates the full cycle. Normal operation flows through the boxes from top to bottom and loops back to idle. Every time the loop restarts, the next target block is determined as the block on the lowest uncompleted structure layer that is closest to the center of the structure and has not been placed yet. This allows us to avoid placing blocks between or below each other, which would risk bumping or interfering with nearby blocks in the structure.

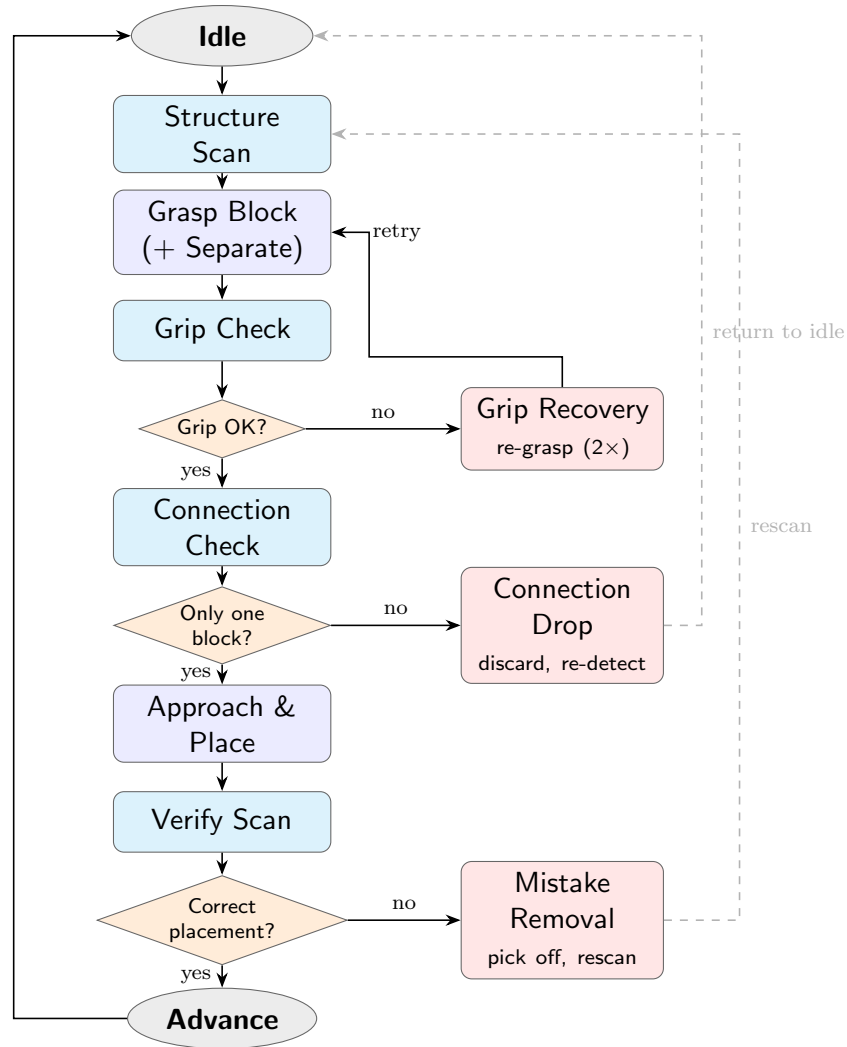


Figure 13: Main build loop. Purple boxes are primary manipulation phases; cyan boxes are scan/sensing phases; red boxes are exception-handling behaviors. After every manipulation phase we have at least one scan/sensing phase to ensure we successfully completed the manipulation phase and to recover if we did not.

8 Shortcomings, Outstanding Items, and Lessons Learned

Although our system is very robust to most scenarios there are several that it cannot handle. The following are the primary limitations of our system:

1. The robot is not physically capable of separating $N \times M$ (with $N, M > 2$) connections of blocks because it cannot grab any of the blocks to separate them.
2. The robot may bump the structure in rare circumstances (but is able to recover from many of these accidents).
3. The robot jerks suddenly if the motors briefly lose connection or the motor commands are sent slightly late.
4. The robot cannot grasp some edge cases of misplaced (either by itself or human placed) blocks.
5. The robot cannot "learn" from its memory/experiences of misgrasped and misplaced blocks.

We have made several algorithm choices to alleviate the affect of these system limitations, but they may still arise. Firstly, the robot will ignore all blocks it has no way to grasp (e.g. blocks inside $N \times N$ connections or outside the workspace) and will only use blocks it can access. To minimize the chance of the robot bumping some part of the structure, we build the structure outwards and upwards as this allows to avoid placing blocks between or below others, which are the biggest causes of the robot damaging the structure. We have also considered (though didn't have time to implement) more significant fixes such as planning a trajectory using Probability Road Maps and A* or some other planning algorithms that would consider the structure as an obstacle. To soften the jerks, we implemented a "catching" behavior, which will recognize when there is a significant divergence in the robot's joint commands and true joint states and will "catch" the robot before it jerks by restarting the current trajectory with the initial position being the true robot joint states rather than forcing the robot to catch up to the trajectory by jerking towards joint states that it is supposed to be in. For misplaced blocks near/on the structure that the robot cannot grasp, it can detect these "unreachable" blocks and signal to the human that it needs help via a nod gesture. Finally, to alleviate the robot's inability to "learn" (or calibrate) kinematics+detection errors on the fly, we implemented an online calibration phase which produces an error map which we use in our ikin solves.

Aside from the initial hardware constraint, the remaining limitations could theoretically be addressed by training and deploying a learned policy. We thoroughly considered this approach—specifically, using reinforcement learning in simulation with domain randomization to train a generalized grasping and placing policy. However, we ultimately concluded that the associated risks and costs (computational overhead, training feasibility, sim-to-real transfer challenges, and extended iteration times) outweighed the likely incremental benefits, especially considering we only had 7 weeks. Through rigorous system engineering, our current rules-based approach already provides a highly robust solution.

9 Appendix A: An example of our structure JSON

We established a generalized JSON format to specify the location of building blocks in the desired structure. The JSON file contains metadata about the requested structure (name, number of blocks, max build grid size and the colors and locations of every block). The following is an example of this JSON format for the flower structure in Figure 14.

```

1  {
2    "metadata": {
3      "version": "1.0",
4      "title": "Flower Block Assembly Instructions",
5      "description": "Instructions for block assembler robot to
6        build 3D structures",
7      "gridSize": { "length": 8, "width": 8, "height": 8 }, "
8        totalBlocks": 23},
9
10   "blocks": [
11     { "id": 4, "color": "green", "position": { "x": 4, "y": 4, "
12       z": 1 } },
13     { "id": 5, "color": "green", "position": { "x": 3, "y": 4, "
14       z": 1 } },
15     { "id": 6, "color": "green", "position": { "x": 5, "y": 4, "
16       z": 1 } },
17     { "id": 7, "color": "green", "position": { "x": 4, "y": 3, "
18       z": 1 } },
19     { "id": 8, "color": "green", "position": { "x": 4, "y": 5, "
20       z": 1 } },
21     { "id": 9, "color": "green", "position": { "x": 4, "y": 4, "
22       z": 2 } },
23     { "id": 10, "color": "red", "position": { "x": 3, "y": 4, "z
24       ": 2 } },
25     { "id": 11, "color": "red", "position": { "x": 5, "y": 4, "z
26       ": 2 } },
27     { "id": 12, "color": "red", "position": { "x": 4, "y": 3, "z
28       ": 2 } },
29     { "id": 13, "color": "red", "position": { "x": 4, "y": 5, "z
30       ": 2 } },
31     { "id": 14, "color": "yellow", "position": { "x": 2, "y": 4,
32       "z": 2 } },
33     { "id": 15, "color": "yellow", "position": { "x": 6, "y": 4,
34       "z": 2 } },
35     { "id": 16, "color": "yellow", "position": { "x": 4, "y": 2,
36       "z": 2 } },
37     { "id": 17, "color": "yellow", "position": { "x": 4, "y": 6,
38       "z": 2 } },
39     { "id": 18, "color": "green", "position": { "x": 4, "y": 4,
40       "z": 3 } },

```

```

24     { "id": 19, "color": "red", "position": { "x": 3, "y": 4, "z": 3 } },
25     { "id": 20, "color": "red", "position": { "x": 5, "y": 4, "z": 3 } },
26     { "id": 21, "color": "red", "position": { "x": 4, "y": 3, "z": 3 } },
27     { "id": 22, "color": "red", "position": { "x": 4, "y": 5, "z": 3 } },
28     { "id": 23, "color": "yellow", "position": { "x": 4, "y": 4, "z": 4 } }
29   ]
30 }

```

Flower Block Assembly Instructions (23 blocks)

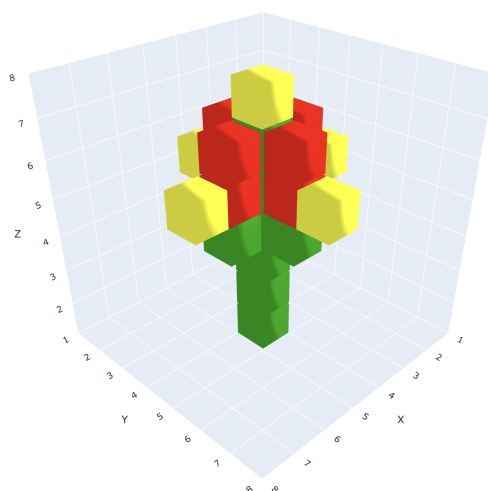


Figure 14: The flower design as specified in the example JSON file.

10 Appendix B: User Interface

10.1 Structure Design Agent

We built a conversational AI agent using the Google Agent Development Kit (ADK) powered by Gemini 2.5 Flash that allows users to design block structures through natural language. The user describes a desired structure (e.g., “build a flower” or “make a 3-layer pyramid”), and the agent iteratively generates, validates, and refines an assembly JSON file. The agent has access to seven tools:

- **get_assembly_constraints** — returns the physical rules the structure must satisfy (grid bounds, cantilever limits, ground connectivity, etc.)
- **count_available_blocks** — reports the inventory of physically available blocks per color

- `list_existing_assemblies / load_assembly` — browse and load previously saved designs
- `verify_assembly` — validates the JSON against eight rules: grid bounds, duplicate positions, duplicate IDs, valid colors, build-order support, cantilever limits, ground connectivity, and reachability
- `visualize_assembly` — renders a layer-by-layer ASCII preview of the structure
- `save_assembly` — persists the verified design to disk as a JSON file

10.2 Dashboard

We implemented a real-time web dashboard served via a standalone FastAPI server. The Brain node posts build-state events (phase transitions, block placements, errors) to the server via a non-blocking HTTP client. The dashboard displays a 3D Three.js rendering of the target structure overlaid with placed blocks, real-time build phase indicators, block placement statistics, an event log with timestamps, and health status indicators for each ROS node. Connected browsers receive live updates via Server-Sent Events (SSE).

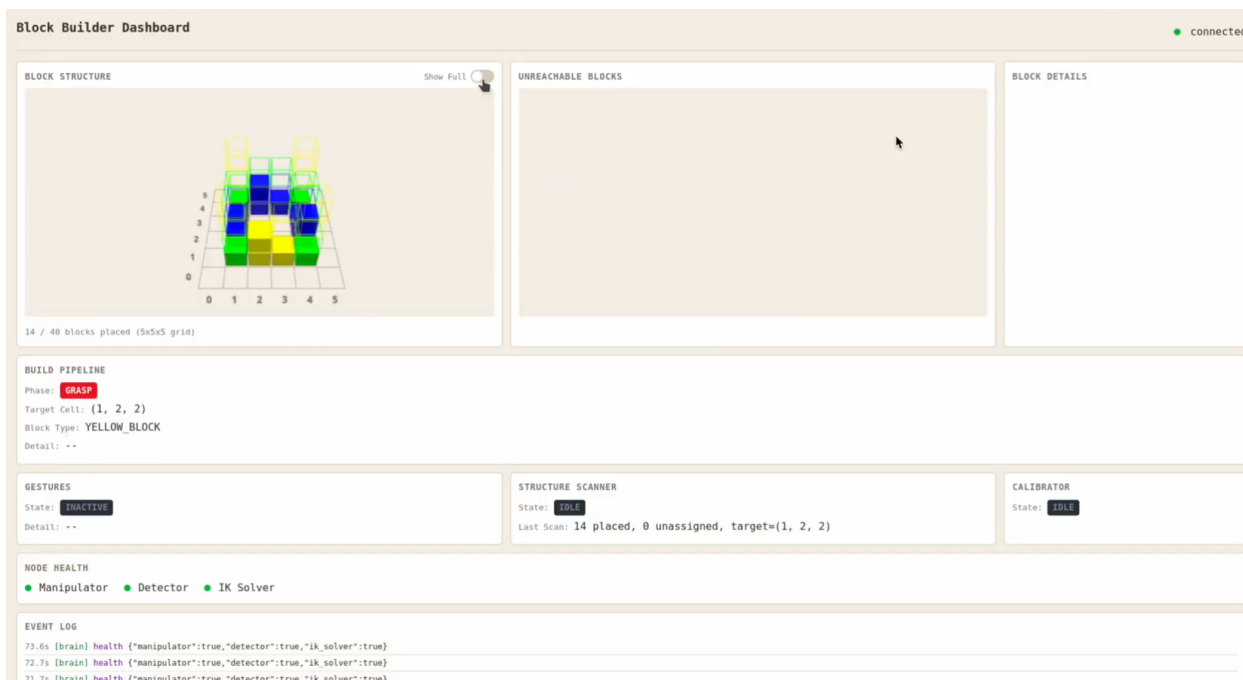


Figure 15: Screenshot of the dashboard.

11 Appendix C: Depth Anything V2 for Grip Analysis

We use the Depth Anything V2 Base model (97M parameters) to produce monocular depth estimates from the end-effector RGB camera. The RealSense depth sensor cannot be used directly for grip analysis because the gripped block is too close to the camera, falling outside

its minimum supported depth range. Instead, we capture an RGB image of the grip area and infer a relative depth map using the pre-trained model.

The model is served via a standalone FastAPI server rather than integrated directly into the ROS detector node. Early experiments showed that running inference within the ROS callback pipeline introduced unacceptable latency, as the model’s GPU compute blocked the detector’s frame-processing loop. Hosting it as a separate HTTP service allows the detector to issue non-blocking requests and continue processing camera frames while depth inference runs in parallel.

Figure 16 shows example depth maps across grip states (good, diagonal, partial_low, partial_high, low) comparing the Small (25M), Base (97M), and Large (335M) model variants. We selected the Base model as it provided sufficient spatial resolution to distinguish grip quality classes while maintaining acceptable inference time.

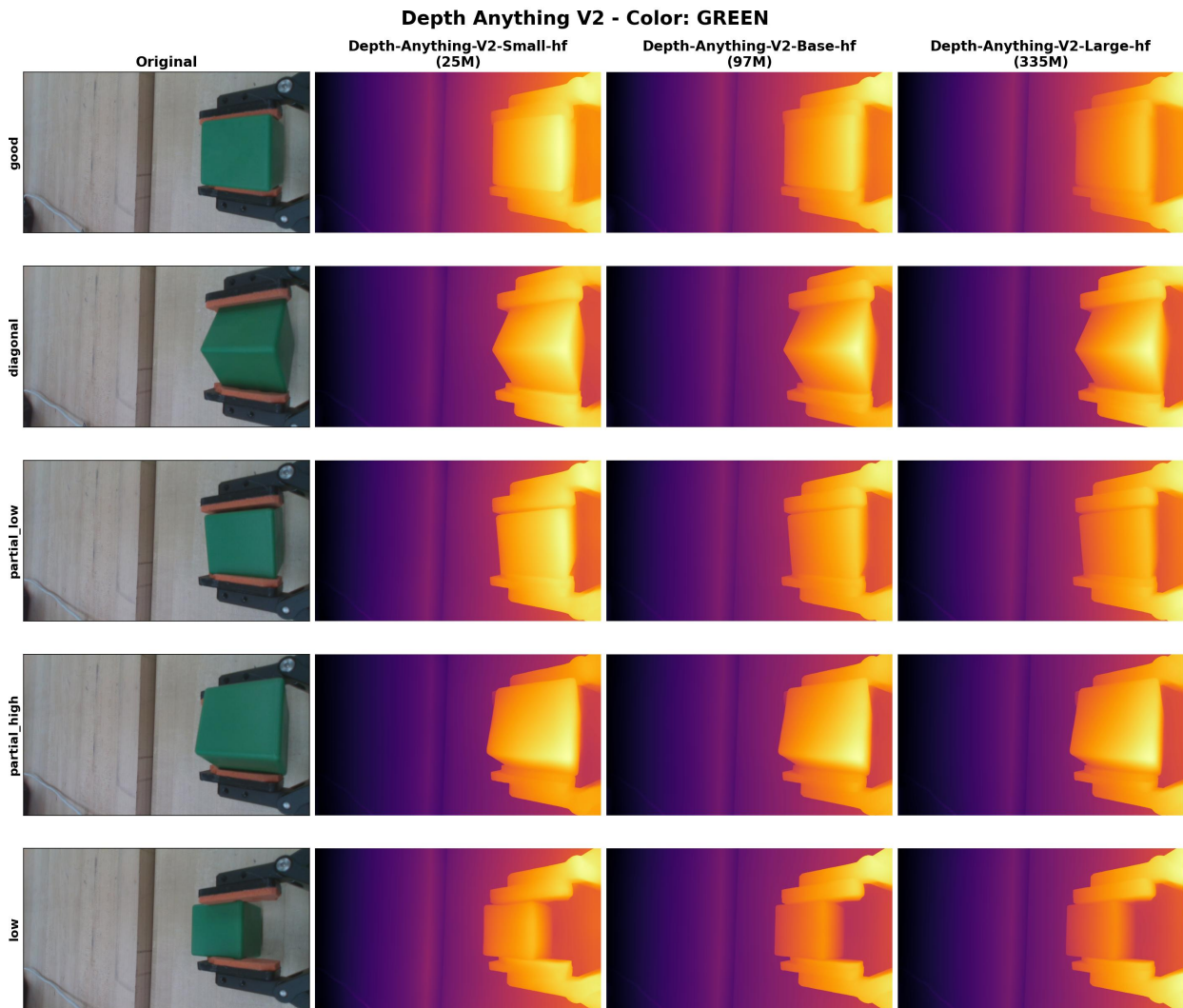


Figure 16: Depth Anything V2 model comparison across grip states. The Base model (97M) provides sufficient resolution for grip classification.